



University of Genoa

Bachelor Degree in Computer Science (L-31)

Evaluation of Metrics for the Detection of Source Code Attacks

Advisor:

Prof. Giovanni Lagorio

Co-Advisors:

Dr. Giacomo Benedetti

Dr. Luca Caviglione

Candidate:

Marco Zoratti

Academic Year 2024-2025

Contents

1	Introduction	1
2	Background	2
2.1	BST Attacks	2
2.2	HUT Attacks	3
2.3	Closing the Gap	5
3	Methodological Approach	5
3.1	Construction of a Dataset Containing Vulnerabilities	7
3.1.1	Collection	7
3.1.2	Backdoor Injection	7
3.2	Methodology	8
4	Study of the Metrics	8
4.1	Analysis and Invalidation of Generic Metrics	9
4.2	Development and Validation of Specific Metrics	10
5	Development of the Solution	12
5.1	Software Architecture	13
5.2	Centralized Configuration System	13
5.3	Implemented Analysis Components	14
5.4	Performance Optimization	14
5.5	Automation of the Analysis Process	14
6	Results	16
6.1	Effectiveness in Detecting BST Attacks	16
6.2	Effectiveness in Detecting HUT Attacks	18
6.3	Analysis of False Positives	19
7	Discussion	20
7.1	Interpretation of the Results	21
7.2	Fragility of the Development Ecosystem	21
7.3	Contributions of the Thesis	22
7.4	Limitations of the Study	22
7.5	Future Work and Practical Implications	23
7.6	Conclusions	24

1 Introduction

Software development is linked to a vast and interconnected ecosystem of components, often published under an open-source license. The modern Software Supply Chain (SSC) is a complex network of dependencies that extends through public repositories specific to the development ecosystem, such as npm (for JavaScript/Node.js), PyPI (Python), and RubyGems (Ruby).

This model, based on code sharing and mutual trust among developers, has accelerated the application creation process and fostered the emergence of innovative solutions and services. However, the distributed nature of modern SSCs has heavily influenced the software’s attack surface, making it more complex and difficult to define.

Attacks that modify source code and hide within it are inherently insidious and are the main topic addressed in this thesis. In this scenario, the attacker does not bypass the development process but exploits it to their advantage with the goal of directly modifying the listing so that the malicious change is accepted and integrated into the main repository through standard contribution channels, such as a Pull Request (PR) on *GitHub*. The code distributed on the package registry will therefore be perfectly identical to the one present in the repository, as the compromise occurred directly at the source level.

In this context, the code review process plays a critical role. Within open-source ecosystems, reviewing changes by project maintainers or other community members is the main quality control mechanism.

When a potential attacker submits a PR, the proposed code is displayed in a *diff* interface, which highlights the changes compared to the current version of the code. The human reviewer is tasked with carefully examining these changes to ensure they are safe, correct, and in line with the project’s goals before approving their integration.

Within this process, it is possible to exploit the human component of the development chain. This thesis aims to analyze not a technical weakness, but the possibility for an attacker to “manipulate” the visual perception of the developer. Specifically, two main families of attacks will be analyzed:

1. **Blank Space Trojan (BST):** This technique allows malicious code to be “appended,” minified into a single line, at the end of a large quantity of spacing characters (spaces or tabs). The goal is to push the harmful payload outside the standard viewing frame of an editor, making it effectively invisible to the reviewer without deliberate horizontal scrolling of the interface.
2. **Hidden Unicode Trojan (HUT):** This category of attacks exploits the immense quantity of characters in the Unicode standard to make the source code be interpreted differently by the compiler than it appears to the human reviewer. The main mechanisms studied in this thesis are:
 - **Visual Similarity via Homoglyphs:** The technique consists of using visually identical letters in function names that belong to different alphabets, such as the Latin ‘a’ and the Cyrillic ‘a’. While the human eye does not notice any difference, the compiler identifies the characters as different: this difference allows a malicious function to be executed in place of the legitimate one without the reviewer being able to notice.

- **Invisible Control Characters:** The technique aims to insert control characters into the code that have no graphical representation but can modify the order of execution of operations, hiding malicious commands within code that at first glance seems harmless.

By exploiting the limits of human perception, attackers can insert backdoors and malicious code that appear innocuous during the code review process. Investigating these attacks is crucial because they undermine the foundation of trust in open-source processes, turning human review into the main attack surface and requiring a new security paradigm.

Given the ineffectiveness of human review alone, the objective of this thesis is the identification and validation of certain descriptive metrics of software artifacts, capable of acting as indicators for BST and HUT attacks. The contribution of this work is therefore in the advancement of Software Supply Chain security, particularly through the analysis of the historical evolution of the code that constitutes a software application.

The rest of the thesis is structured as follows: Section 2 provides the necessary background, illustrating examples of real attacks. Section 3 describes the methodological approach, from the creation of the dataset to the definition of the applied methodology. Section 4 analyzes the software metrics in detail, validating the most effective ones. Section 5 describes the technical architecture of the developed tool. Finally, Section 6 presents and analyzes the results obtained, including a discussion of false positives.

2 Background

In this section, some examples of BST (Section 2.1) and HUT (Section 2.2) attacks will be presented.

2.1 BST Attacks

The first example relates to the *colorama* package in the Python ecosystem, a cross-platform library that allows for coloring textual output displayed through the terminal. In this case, the attackers managed to distribute a malicious version of the package thanks to a *typosquatting* attack and malicious code hidden via BST, as shown in Figure 1.

First, the attackers created a fake mirror site to distribute a harmful version of the package. Subsequently, after compromising the accounts of some developers, they modified the *top-gg* project to have it download *colorama* from their malicious site. Finally, the harmful code inside the package was hidden via BST, that is, concealed by a large amount of whitespace.

This case study demonstrates how a relatively simple visual obfuscation technique like BST can be integrated into an attack on the SSC, combining social engineering and typosquatting to maximize impact [1].

The second example relates to a serious security incident that affected the popular *rand-*

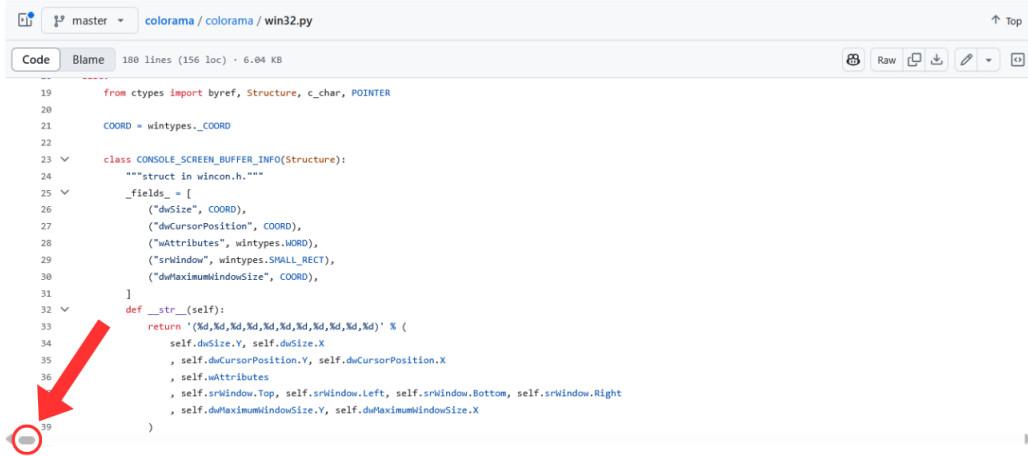


Figure 1: Simulation of the BST attack on the *colorama* package. The malicious code is hidden off the visible screen due to whitespaces

user-agent package on npm, used to distribute a sophisticated remote access trojan.

The attackers stole developer accounts and published numerous malicious versions, hiding the malicious code via BST, as shown in Figure 2.

This case exemplifies an attack on the deployment infrastructure, where the source code on *GitHub* remained clean, but the package distributed on npm was compromised.

The threat was not discovered by a human review, but by an automated security scanning tool (Aikido Security [2]) that statically analyzed the package and detected the obfuscated code, but not the concealment. Subsequently, a human intervention brought the perfectly hidden and functional attack to light.

2.2 HUT Attacks

The first type of HUT attacks analyzed are *Trojan Source* [5]. These threats cause the order of the source code to appear different to the reviewer than how it is read by the compiler. To this end, the attack exploits Unicode directional control characters, called *BiDi* [12].

Figure 3 shows how a *Trojan Source* attack works:

- Deceptive Appearance (Figure 3a): The code appears normal and safe to the human reviewer’s eye.
- Real Content (Figure 3b): The source file actually hides invisible control characters that alter the visual order of the text.
- Malicious Execution (Figure 3c): Compilers and interpreters ignore the *BiDi* characters, reading the code in its logical and harmful order, executing the attack.

The result is that a malicious line of code can be hidden or made to appear as a harmless comment by swapping the order of words, deceiving a human reviewer. This vulnerability is present in almost all modern programming languages and, at the time of its discovery,

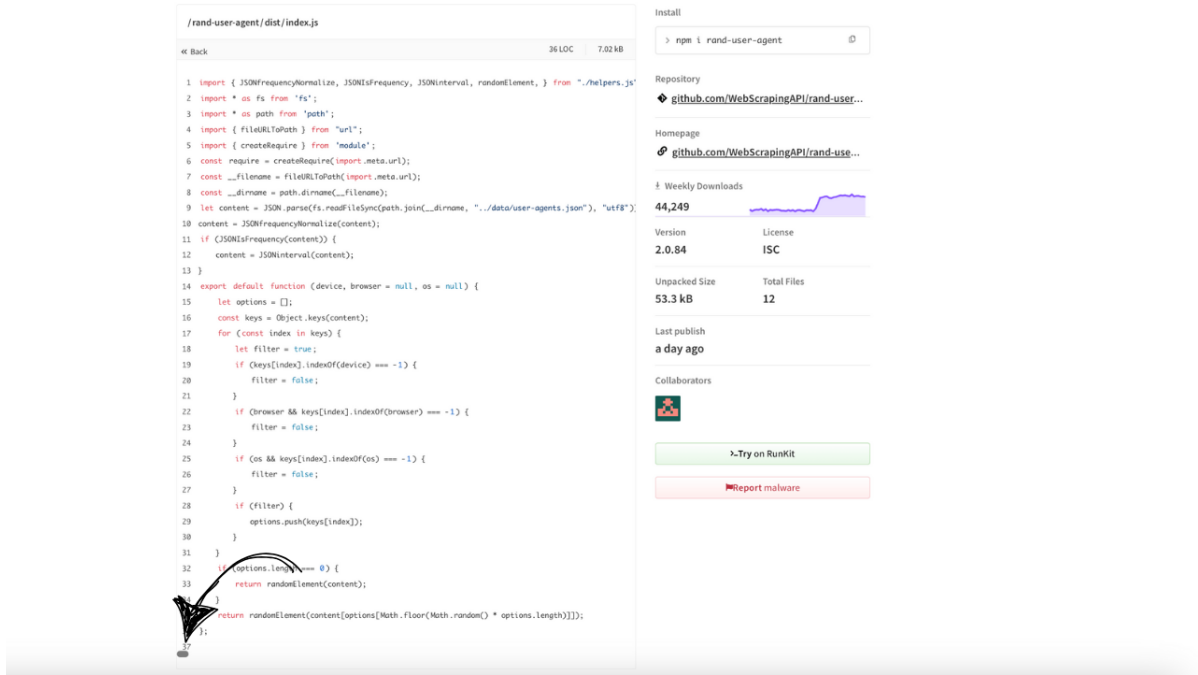
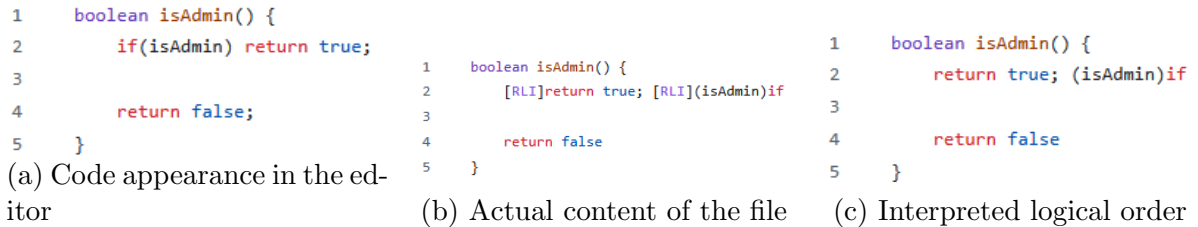


Figure 2: Code hidden in the npm package *rand-user-agent*.

Credits: https://www.aikido.dev/blog/catching-a-rat-remote-access-trojan-rand-user-agent-supply-chain-compromise (https://www.aikido.dev/blog/catching-a-rat-remote-access-trojan-rand-user-agent-supply-chain-compromise)

Figure 3: Examples of *Trojan Source* attacks



was effective on the most popular code editors and platforms like *VS Code*, *GitHub*, and *GitLab* [4].

Another form of HUT attack is the homoglyph attack, a visual deception technique that exploits the existence of Unicode characters that, while appearing identical, are encoded differently by the compiler (e.g., the Latin letter 'a', encoded as U+0061, and the Cyrillic 'a', encoded as U+0430).

The mechanism is as follows:

1. Homoglyph Character Insertion: An attacker defines a malicious function or variable using a name that contains a homoglyph character, making it visually identical to a legitimate function. For example, a harmful function called `is_valid()` (with the Cyrillic 'a') is indistinguishable from the safe function `is_valid()`.
2. Unaware Execution: A developer using the infected library can call the malicious

function without noticing. During code review, the deception is very difficult to spot, as the function name appears visually correct.

In essence, the homoglyph attack is a form of “typosquatting” that occurs directly within the source code, rather than at the package name level. This makes it a more stealthy and difficult-to-detect threat during the normal development and review process [6].

To exemplify a manipulation via HUT, consider the scenario illustrated in Figure 4. The images show the screen of a Pull Request (PR) that, at first glance, seems to contain only minor and legitimate changes. In reality, in Figure 4, the call `ask_personal_name` on line 24 contains a Cyrillic ‘a’ which prevents the program from calling the legitimate function on line 5 and instead calls a malicious one, defined in a dependency or another file.

This example demonstrates how attackers can obfuscate their attack by exploiting code review interfaces and the limits of human perception, transforming the reviewer into the weak point of the security chain.

2.3 Closing the Gap

This study aims to focus on a new emerging class of attacks that do not exploit software flaws or bugs but the developers themselves. Studies like “Trojan Source” [5] have demonstrated how Unicode encoding can be used to create code whose appearance masks malicious logic.

Despite awareness of these threats, current defense tools are insufficient: SAST tools are ineffective, as they examine the logical structure of the code, i.e., the Abstract Syntax Tree (AST), but not its graphical representation.

Likewise, development platforms like *GitHub* have implemented partial countermeasures, which, however, do not cover all attack variants and are not always active.

These shortcomings, combined with the lack of public datasets of attacks attributable to the BST and HUT types, made it necessary to develop a new analysis methodology based on the historical analysis of projects. The creation of a dataset with injected vulnerabilities was an essential step to be able to test and demonstrate the effectiveness of this approach.

3 Methodological Approach

This section presents the methodological approach adopted in this thesis. The first phase involved the construction of an appropriate dataset (Section 3.1), while the second phase focused on defining the source code analysis methodology (Section 3.2).

3.1 Construction of a Dataset Containing Vulnerabilities

In order to conduct an effective analysis, it was necessary to create a dataset containing projects that included BST and HUT type vulnerabilities. This step proved to be a mandatory choice, as the current scientific literature does not provide public datasets useful for the investigation planned in this thesis work.

The generation of the dataset containing the vulnerabilities required the selection of public repositories and the subsequent injection of a backdoor.

The implemented process allowed for simulating the action of an attacker against a specific repository in a controlled environment. The compromised code was in no way proposed on the original repositories, remaining available only locally.

3.1.1 Collection

Starting from a broader list of popular JavaScript projects, 91 repositories were selected that fully met the following inclusion criteria:

1. **Minimum Number of Tags:** each repository must have at least 10 tags, so at least 10 releases of successive versions that follow the standard of *semantic versioning* [9].
2. **Project Size:** Based on quantitative and qualitative metrics, the 91 repositories in our study were divided into two categories:
 - **'Large' Projects** (61 repositories). Mature projects that meet the following criteria: a team with over three active contributors, an extended history with hundreds of releases, and indicators of frequent maintenance.
 - **'Small' Projects** (30 repositories). Single-author projects, whose distinctive characteristics are a uniform development style and a software lifecycle managed individually.

The list of *GitHub* repositories examined is available at the link [https://github.com/marchacio/BST_HUT_analyzer/blob/main/run/const/repo_list.py] (https://github.com/marchacio/BST_HUT_analyzer/blob/main/run/const/repo_list.py)

3.1.2 Backdoor Injection

To speed up the dataset creation process, an automated procedure was implemented to inject a backdoor into the repository under examination.

Initially, the process identifies the latest stable version of the repository, which is the last tag. Subsequently, it randomly selects a source code file from the project (excluding minified, configuration, or test files) and adds a backdoor hidden using the BST and HUT offensive paradigms.

Finally, the modification is saved by creating a new commit and marking this compromised version with a special tag named "TEST".

This procedure was implemented through a Python script and was repeated for all 91 projects under analysis. To make the dataset creation more realistic, it was decided to inject a backdoor structured as follows:

```

1 var https = require('https');
2 https.get('[https://www.fakemaliciouswebsite.it/fake-
  vulnerability-download](https://www.fakemaliciouswebsite.it/
  fake-vulnerability-download)',
3   res => res.forEach(
4     chunk => eval(chunk.toString())));

```

Its operation involves importing the *https* module of *Node.js* on line 1; subsequently (lines 2-4), a GET request is made to a remote server and the content of the response is executed directly via `eval()`, thus completing the attack. Furthermore, this code is obfuscated using the `fromCharCode()` function, a common technique for hiding malicious strings. The final result will therefore be composed as follows:

```

1 eval(String.fromCharCode(118,97,114, ... , 41,41,59));

```

3.2 Methodology

The approach for building the analysis tool is mainly based on one concept: vulnerabilities like BST and HUT are introduced at a precise moment in a software's life and leave a measurable trace. Consequently, the most effective detection does not come from a static inspection of a single version of the code, but from a historical analysis of its evolution.

The operational principle is to treat the sequence of releases of a Git repository (identified by tags) as a time series in which, for each release, the tool calculates a “metric profile” of each source file.

The vulnerability is therefore not detected by the absolute value of a metric, but by the presence of anomalies in its variation between one version and the one immediately following it. A sudden and statistically significant growth in the value of a metric for a given file is a warning sign, suggesting potential tampering occurred in that specific transition.

Managing false positives represents one of the main challenges of this approach. Given that even legitimate and large-scale changes can cause metric spikes, it becomes crucial to calibrate the alert thresholds to minimize false positives. The goal is to ensure that only the most suspicious deviations are flagged, distinguishing them effectively from the normal evolution of the software.

4 Study of the Metrics

This section presents the logical process of selecting and validating the metrics used to detect BST and HUT type attacks, highlighting not only the techniques that have proven effective but also those that were discarded.

4.1 Analysis and Invalidation of Generic Metrics

The initial phase of the research focused on the application of standard metrics, widely used in software engineering literature to evaluate project quality, maintainability, and evolution. The starting hypothesis was that the introduction of an attack into the source code could generate measurable perturbations even at a macroscopic level. However, this hypothesis proved unfounded for the type of attacks considered.

The first metrics analyzed were those related to code size:

- **Lines of Code (LOC):** Useful for tracking a project’s growth, but ineffective in our context. A BST or HUT attack can be structured on a single line of code, even by modifying an existing one. In a project with 50,000 LOC, the addition or modification of one line represents a 0.002% variation, a signal indistinguishable from the “noise” generated by normal development activity.
- **Code Size in Bytes:** Although more sensitive than LOC, this metric is also easily fooled. A legitimate change such as adding a dependency can increase the project’s size by several kilobytes or megabytes, completely masking the increase of a few bytes or kilobytes due to the insertion of a malicious payload. Furthermore, its dependence on language verbosity and formatting makes it an unstable indicator.

A second approach involved analyzing repository metadata and temporal patterns, assuming that an attacker’s action could deviate from normal development routines.

- **Commit Frequency and Times:** It was hypothesized that a malicious change could occur at unusual times compared to the average. This metric proved unreliable: in international open-source projects, there can be contributors from different time zones who therefore operate on the code at all hours of the day. Moreover, an attacker can easily manipulate a commit’s date to make it appear normal to a reviewer.
- **Release Analysis:** The analysis of the release type (major, minor, patch), their frequency, and the authors proved useless. An attacker can mask the injection of malicious code within any type of release and by adapting to the project’s development rhythm.

Having ascertained the ineffectiveness of macroscopic indicators, the focus shifted to more sophisticated metrics, capable of analyzing the intrinsic structure of the code.

- **Cyclomatic Complexity:** This metric quantifies the complexity of a program’s control flow and is excellent for identifying intricate code. However, a payload can be very simple and adds at most a single node to the program’s control graph. Furthermore, the attacks studied in this thesis do not exploit logical complexity, but lexical and visual obfuscation.
- **Shannon Entropy:** Entropy measures the amount of information contained in a data stream and is often used successfully to detect compressed or encrypted code, which tends to have very high entropy. In our case, this metric fails for opposite reasons: a BST attack injects an extremely long and repetitive sequence of the space character, which has an entropy close to zero. The addition of this sequence

to a source code file lowers its average entropy, a signal that could be interpreted incorrectly. For HUT attacks, the introduction of a few Unicode characters into a file of hundreds or thousands of lines is too small a perturbation to be detected by a measure calculated on the entire file.

Limits of SAST Analysis Finally, the use of *Static Application Security Testing* (SAST) tools was evaluated. These tools look for code patterns corresponding to known vulnerabilities, but this approach presents two problems in the context we analyzed:

1. **Attack Obfuscation:** Attackers constantly develop new obfuscation techniques precisely to evade the searches of SAST tools.
2. **Context:** The fundamental problem is that SAST tools analyze the code’s AST but are not designed to see and evaluate the code like a human being. Consequently, they do not possess mechanisms to detect anomalies in the distribution of whitespaces, nor to identify the deceptive use of homoglyph or text directionality control characters, which are the bases of BST and HUT attacks.

4.2 Development and Validation of Specific Metrics

Since the generic metrics from Section 4.1 proved ineffective, a change in approach was necessary. The analysis therefore focused on creating specific indicators for BST and HUT attacks, examining the code at the single-line or character level rather than the file level. In fact, the *modus operandi* of a BST attack is the injection of a massive number of whitespaces to push the payload out of the human programmer’s editor view; This behavior leaves clear statistical traces.

- **Maximum Line Length (MLL):** A direct side effect of the attack is the creation of an extremely long line of code. Tracking the maximum length of a line for each file across the various versions is an effective method. For example, a line that goes from 80 characters to 2080 is an anomaly impossible to ignore. The limitation of this metric emerges in files already characterized by long lines, such as minified files.
- **Whitespace to Printable Character Ratio (W/P Ratio):** Among all the metrics analyzed, this proved to be the most powerful and direct for identifying BST attacks. Its logical foundation is very simple: every code file has a relatively stable balance between the characters that constitute its logic and the whitespaces used for formatting and readability. A BST attack unequivocally alters this balance, adding a significantly large number of empty characters.

During the study of this metric, two possible formulations were considered:

1. **Total proportion:** The first option calculates the percentage of whitespace relative to the total number of characters in the file.

$$Ratio_A = \frac{W}{P + W}$$

Where W is the number of whitespace characters and P is the number of printable characters. Let's assume we have a file with 800 characters and 200 whitespaces that undergoes a BST attack with the addition of 3000 spaces and a payload of 500:

– *Normal Scenario:*

$$Ratio_A = \frac{200}{800 + 200} = 0.2 \quad (20\%)$$

– *Attacked Scenario:*

$$Ratio_A = \frac{200 + 3000}{(800 + 500) + (200 + 3000)} = 0.711 \quad (71.1\%)$$

This metric works and is intuitive, but the signal is mathematically dampened by the increase in the denominator.

2. **Direct Ratio (Adopted Choice):** The second option calculates the direct ratio between whitespaces and only printable characters.

$$Ratio_B = \frac{W}{P}$$

– *Normal Scenario:*

$$Ratio_B = \frac{200}{800} = 0.25 \quad (25\%)$$

– *Attacked Scenario:*

$$Ratio_B = \frac{200 + 3000}{800 + 500} = 2.46 \quad (246\%)$$

This metric produces a much more “explosive” signal, with a value that increases by about 10 times in the example scenario.

The second formulation ($Ratio_B$) was therefore chosen, as it maximizes the metric's sensitivity for detecting anomalies. This creates a much cleaner and more isolated signal, making the detection system more reliable.

HUT attacks are more subtle because they manipulate the graphical representation of the text, and their detection requires a single-character level inspection, searching for specific Unicode codes.

- **Homoglyph Character Count:** The strategy consists of creating a list of known homoglyphs (e.g., Cyrillic, Greek, or Latin characters that resemble ASCII characters) and scanning the source code for their presence. Although the occasional use

of non-ASCII characters may be legitimate (e.g., in strings or comments), their presence within variable identifiers or function names is highly suspicious. One attack scenario could be:

```
1 // Legitimate function
2 function isUserAdmin(user) { /* ... */ }

1 // Malicious function with homoglyph (with the Cyrillic 'А')
2 function isUserAdmin(user) { /* malicious code */ }
```

In fact, an attacker could replace a call to the legitimate function with the malicious one, and the change would be almost invisible in a code review. The developed method is based on counting these suspicious characters, flagging files that have one or more occurrences or an increase.

- **Invisible Character Count:** This is the most specific and direct metric for *Trojan Source* [5] type attacks. The procedure consists of scanning the code for specific Unicode characters that alter the source interpretation logic. Among these, the most dangerous are the *BiDi* characters [12], such as 'U+202E' (Right-to-Left Override), whose original purpose is to include alphabets that read from right to left like Arabic and Hebrew. The developed tool keeps a count of these characters and flags their presence anywhere in the code, as their use in a programming file is almost always illegitimate.

In conclusion, the methodological path has shown that to counter specific and unconventional threats like BST and HUT, generic software metrics are insufficient. A targeted approach was necessary, based on a deep understanding of the attack techniques, which led to the development of fine-grained indicators.

5 Development of the Solution

This section describes the architecture, implementation, and evolution of the software tool developed for the analysis and detection of BST and HUT vulnerabilities. The tool, named *BST_HUT_analyzer* and available on *GitHub* at [https://github.com/marchacio/BST_HUT_analyzer] (https://github.com/marchacio/BST_HUT_analyzer), is written in Python and implements the metrics identified in section 4.2.

The tool was designed to systematically analyze the history of a Git repository, identifying anomalies at the level of individual releases (tags).

Specifically, the following key aspects of the development process will be explored in depth: Software Architecture (Section 5.1), Configuration System (Section 5.2), Analysis Components (Section 5.3), Performance Optimization (Section 5.4), and Process Automation (Section 5.5).

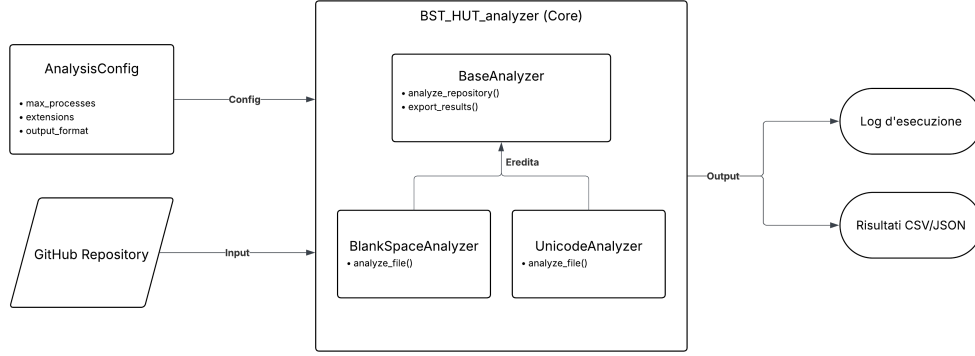


Figure 5: *Pseudo* Class Diagram of the project

5.1 Software Architecture

As shown in Figure 5, the core of the project is the abstract base class **BaseAnalyzer**, defined in the file `core/analyzer.py`. This class acts as a “template” for creating new analysis modules, defining a common interface and managing basic operations.

The responsibilities of the **BaseAnalyzer** class are:

- **Analysis Management:** The `analyze_repository` method iterates over all tags of a repository, ordering them chronologically. For each tag, it checks out the corresponding version of the code and starts the analysis process for the selected tag.
- **File Discovery:** Through the `_get_files_for_extension` method, the system recursively searches for all files with a specific extension (e.g., `.js`), while excluding irrelevant directories (such as `node_modules` or `.git`) to reduce noise and overhead.
- **Result Management:** The base class defines a standard data structure for results, using the `FileAnalysisResult` and `TagAnalysisResult` dataclasses to aggregate metrics, anomalies, and metadata.
- **Standardized Export:** The `export_results` method manages saving the collected data in formats such as JSON and CSV, delegating the specific CSV format implementation to the subclasses.

This design allows the tool’s capabilities to be extended by introducing new analyzers (e.g., for a new class of vulnerabilities) without modifying the central architecture, but simply by creating a new class that inherits from **BaseAnalyzer**.

5.2 Centralized Configuration System

A centralized configuration system was implemented to manage the tool’s behavior flexibly and without the need to modify the source code. The solution is based on the **AnalysisConfig** class, defined in the file `config.py`, which groups all parameters that influence execution, such as `supported_extensions` to configure the file extensions to be examined or `filter_dirs` to ignore folders and subfolders, such as `node_modules` or `dist`.

Regarding performance, the `max_processes` parameter allows adjusting the number of parallel processes to be used, enabling performance optimization based on available hardware. Finally, for the output, you can choose the result format (`output_format`) between CSV and JSON and define the path for log files (`log_file`). A key feature of the `AnalysisConfig` class is the `__post_init__` method, which sets intelligent default values if none are explicitly provided.

5.3 Implemented Analysis Components

Leveraging the abstract base architecture, two concrete analyzers were implemented: `BlankSpaceAnalyzer`, located in `core/blank_space_analyzer.py`, which implements the metrics discussed in Section 4.2 for identifying BST attacks.

Similarly, `UnicodeAnalyzer`, located in `core/unicode_analyzer.py`, was developed to counter HUT attacks. To improve efficiency, the analysis is not performed on every single character of the file, but the frequency of each unique character is calculated via `collections.Counter`, and the analysis is then performed only on this reduced set of characters.

5.4 Performance Optimization

In the initial development phases, file analysis was performed in a purely sequential manner. Although functional, this approach proved to be a bottleneck during the analysis of large repositories, as the analysis time for a single repository could take several hours, limiting the feasibility of large-scale experiments.

To overcome this limitation, support for parallelization was introduced thanks to Python's *multiprocessing* library.

However, an important trade-off related to this optimization emerged during the testing phase. For medium-to-small sized repositories, the computational overhead associated with creating, synchronizing, and closing the process pool proved to be greater than the gain obtained. In these scenarios, sequential analysis was actually faster and more efficient.

The tool was configured to allow the user to choose the most suitable strategy for the context: the parallel mode guarantees the scalability needed to handle enterprise-level projects, while the sequential one remains the most performant option for quick analyses on smaller projects.

5.5 Automation of the Analysis Process

To ensure repeatability and consistency in large-scale data collection, the repository analysis process was fully automated through a system of dedicated scripts that follow the guidelines specified in the *activity diagram* shown in Figure 6.

The system is based on two main components: a centralized list of target repositories, defined in the file `run/const/repo_list.py`, and a series of execution scripts

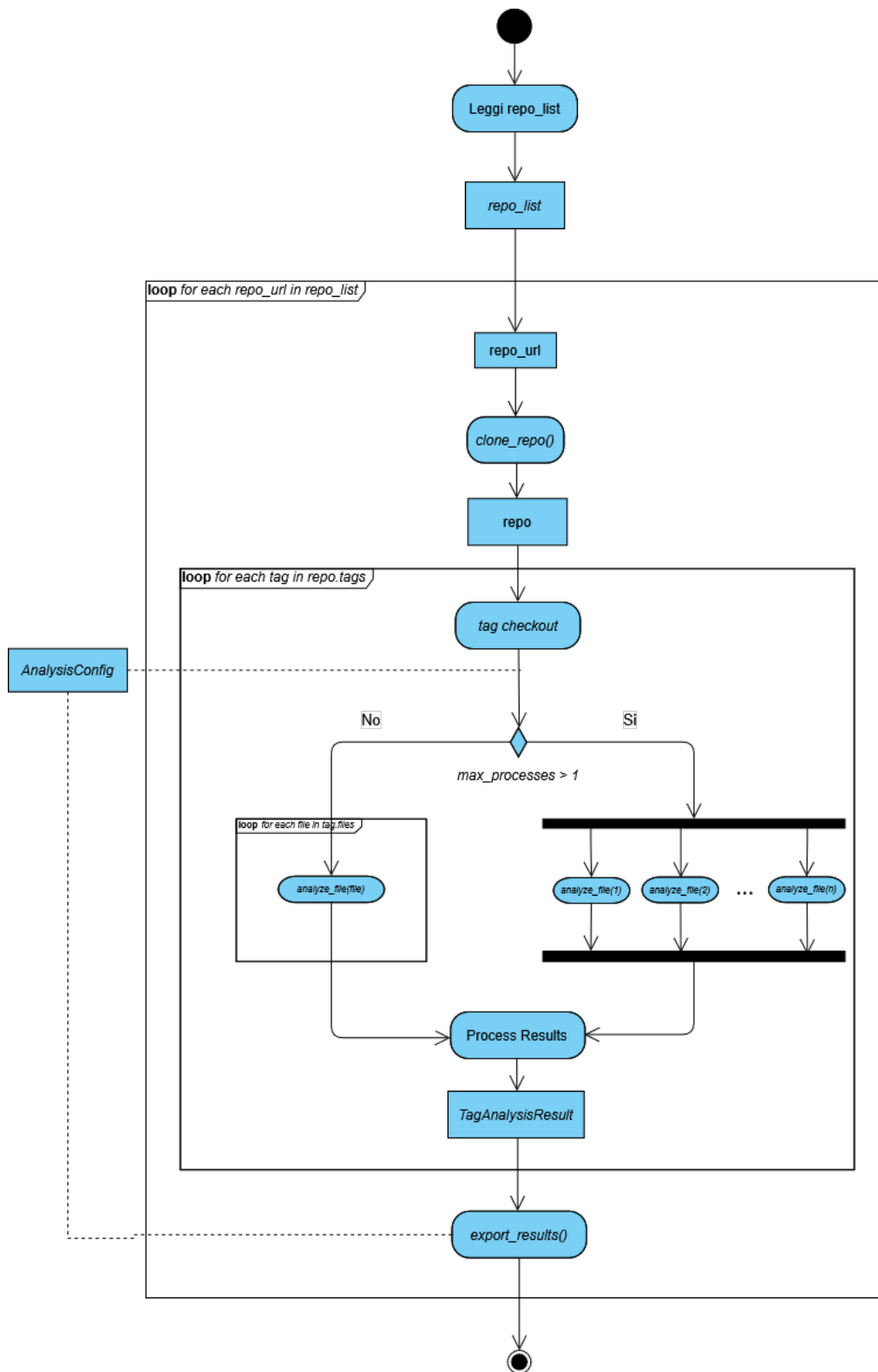


Figure 6: Activity Diagram of the dataset analysis

(`original_blank_space_run.py`, `manipulated_blank_space_run.py`, `unicode_run.py`) that manage the entire workflow.

Each script iterates over the list of repositories, configures the specific analyzer (`BlankSpaceAnalyzer` or `UnicodeAnalyzer`), clones the source code, performs the analysis, saves the results in CSV format, and a log file to track the execution progress.

6 Results

This section presents the results obtained from applying the tool to the dataset described in Section 3.1. In particular, Section 6.1 presents the effectiveness of BST attack detection, while Section 6.2 focuses on HUT attacks. For both types, the results obtained from the analysis of four representative projects of common development methodologies in the open-source context are analyzed.

In detail, two of the projects (*Angular* [3] and *Vue* [11]) have a large number of contributors, intense development activity (more than 50 commits per week), and a large amount of code (1,000,000 LoC on average).

Conversely, two of the selected projects (*Superstruct* [7] and *Redom* [10]) have a small number of active contributors (3 or less), reduced development activity (less than 10 commits per week), and a limited amount of code (7500 LoC on average). Finally, Section 6.3 analyzes the causes of the false positives obtained during the conducted experiments.

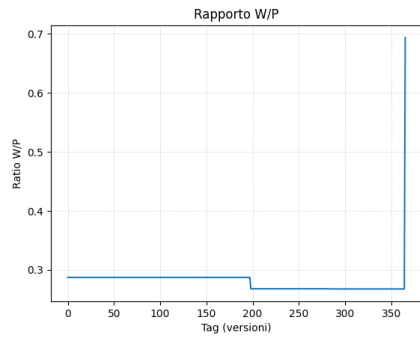
6.1 Effectiveness in Detecting BST Attacks

The proposed solution proved effective in identifying the presence of BST attacks. In particular, Figure 7 clearly shows how the W/P ratio is effective both in modeling the software lifecycle and in showing any anomalies.

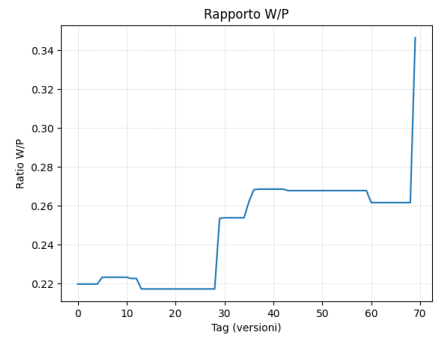
Specifically, in the case of development processes that cause a non-constant trend of this metric, such as *Vue*, it is possible to clearly identify the anomaly caused by the attack. In fact, in this case, *Vue* shows a first increase of 21% in the metric between the releases *2.6.14* and *2.7.0-alpha.1* (the 28th and 29th releases respectively for the file being analyzed).

This increase is due to the introduction of new features and significant structural changes in the minor release, which led to a legitimate increase in whitespaces and code formatting. However, the increase is less evident than that caused by the payload insertion (35%). This metric proves to be particularly effective in the case of an intense development process like that of *Angular*, where the large number of releases (>300) makes the anomaly even more evident.

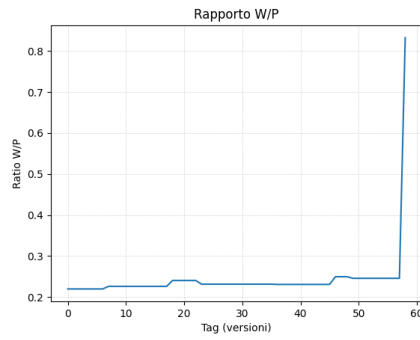
The metric also manages to be effective for the projects selected to represent less intense development methods, *Superstruct* and *Redom*. In particular, on these projects, a less constant trend is noted in the period preceding the payload insertion, probably due to less constant changes over time and therefore more visible (e.g., a single developer inserts



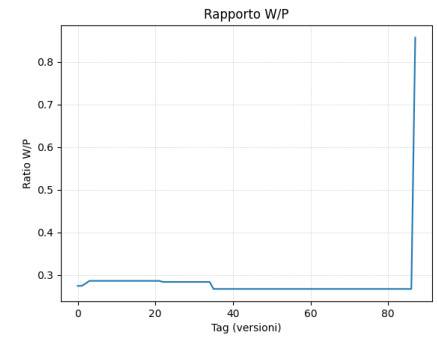
(a) *Angular*



(b) *Vue*



(c) *Redom*



(d) *Superstruct*

Figure 7: Four graphs showing the temporal trend of the W/P Ratio metric for the repositories examined.

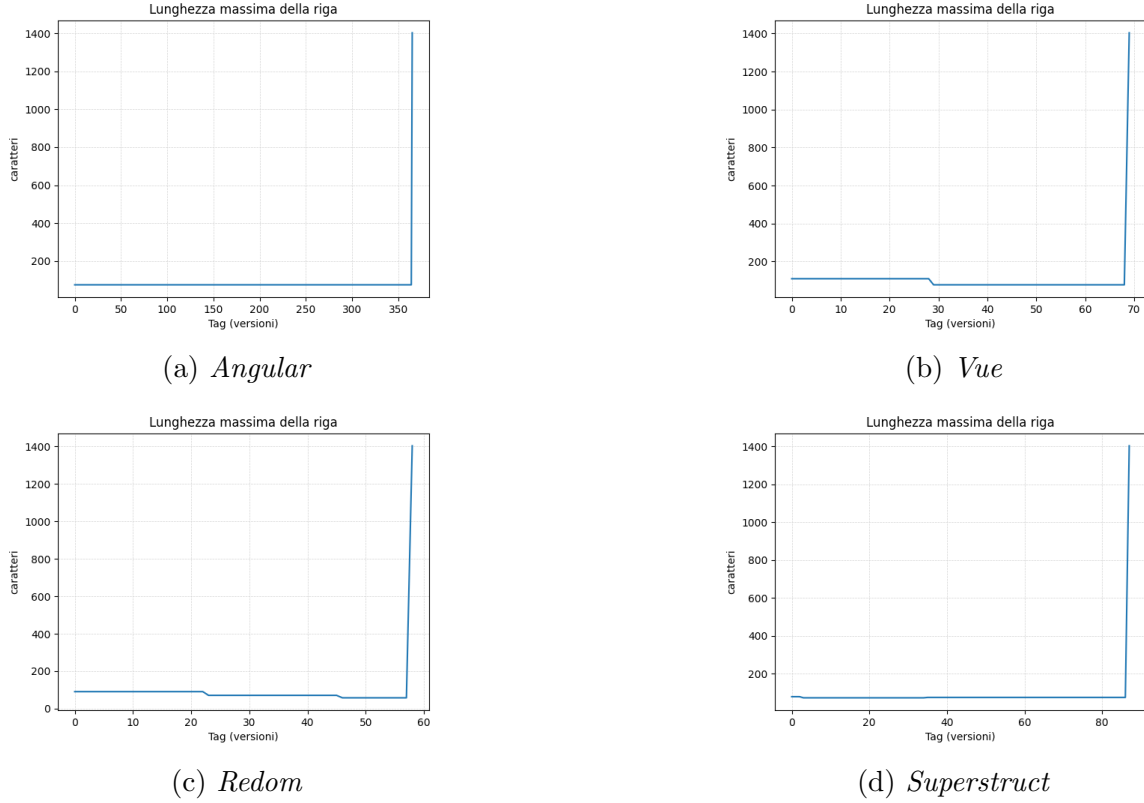


Figure 8: Four graphs showing the temporal trend of the MLL metric for the repositories examined.

considerable amounts of code after long time intervals, this is captured more visibly by the metric).

Figure 8 shows the behavior of the MLL metric. It is possible to note that it too is capable of capturing the presence of the malicious payload within the code. However, it is not particularly representative of the software lifecycle, as it maintains an almost constant value for all the projects analyzed.

This data is strongly influenced by common standard practices among developers, who tend not to exceed 80 characters per line [8]. Furthermore, as will be detailed later, this metric captures an instantaneous characteristic of the code, which could be altered by common causes.

An example of these possible alterations is the case of particularly complex lambda functions, which have the possibility of being written inline in most programming languages.

6.2 Effectiveness in Detecting HUT Attacks

The proposed solution proved effective in identifying the presence of HUT attacks. Our analysis revealed that homoglyph characters are extremely rare within the examined projects: out of a total of 41,903 tags and 91 repositories, their presence was found in only 156 versions, spread across 10 projects, equal to 0.3% of the tags analyzed. This characteristic allows for effective identification of HUT attacks by directly observing the presence or insertion of homoglyph characters. Taking into account the four projects

Table 1: Variation of HUT metrics for the files of the repositories examined.

Project	Tag	Homoglyph Count	Total Characters
<i>Angular</i>	...	0	...
	v14.2.9	0	3589
	v14.2.10	0	3589
	v14.2.11	0	3589
	TEST	4	4683
<i>Vue</i>	...	0	...
	v3.2.43	0	8732
	v3.2.44	0	8822
	v3.2.45	0	8822
	TEST	4	9916
<i>Redom</i>	...	0	...
	v3.25.0	0	3281
	v3.26.0	0	3281
	v3.27.0	0	3310
	TEST	4	4404
<i>Superstruct</i>	...	0	...
	v0.6.0	0	696
	v0.6.1	0	696
	v0.7.0	0	696
	TEST	4	1790

already considered for BST attacks, Table 1 shows that the files in which the malicious payload was inserted do not contain any homoglyph characters throughout their lifecycle. However, in the “TEST” release, in which a backdoor hidden via HUT was added, a suspicious and anomalous increase in the number of homoglyph and total characters can be noted, an unmistakable sign of tampering by an attacker.

Although in this specific test scenario the two metrics vary together, it is essential to emphasize that file size is not a reliable indicator for HUT attacks. An attacker could replace a single legitimate character with its homoglyph without altering the total length of the code, making the attack invisible to an analysis based on size alone; therefore, the fact that the total number of characters remained constant for numerous versions is not a sufficient condition. The real alarm signal is the binary nature (presence/absence) of the suspicious characters, which makes it a high-fidelity alarm signal that allows the threat to be isolated with precision.

6.3 Analysis of False Positives

A fundamental aspect of evaluating the effectiveness of the selected metrics is the ability not to generate an excessive number of false alarms. To identify the causes of false positives, the tool was run on the dataset excluding the version in which the malicious payload was inserted. Each anomaly was considered a false positive and analyzed to

classify its origin.

In the case of BST attacks, the main cause lies in the presence of minified files within the project. When a developer creates a new version of the project by adding minified assets, test files, or configuration files, the selected metrics are invalidated. In particular, the W/P Ratio is influenced by the removal of whitespaces applied by the minification process, while the maximum line length is rendered superfluous by the removal of line breaks present in the code.

It is worth noting that the minification process also influences the success of the attack, as it drastically changes the code visualization. Once minified, the payload could appear clearly to the developer or be rendered incomprehensible, depending on the minification process used.

However, the presence of false positives can be mitigated with simple modifications to the approach used. In detail, the current implementation of the tool applies a filter to ignore some files based on their extension (e.g., `.min.js`, `.min.css`, `.map`) and to exclude some commonly used directories from the analysis as destinations for build processes or for third-party libraries (e.g., `dist`, `build`, `vendor`, `assets`) that do not constitute an attack surface for the types of malicious code analyzed in this thesis.

Likewise, we can identify the causes of false positives during the identification of HUT attacks. In particular, within our dataset, we have identified three relevant causes: mathematical symbols, localization files, and potential copy-pastes.

The first are present particularly in projects that contain functionalities related to mathematical operations where developers have inserted comments describing the implemented functions. The second concerns projects where text localization (translation) is offered for some graphical components, which implies the presence of characters with a different encoding. The last cause is due to copy-pasting from files in other formats, presumably PDF, which introduce non-visible characters.

These types of false positives can be mitigated by implementing an approach linked to the program's AST. This would allow the analysis to be focused only on components deemed susceptible to attack, such as function or variable names.

Another approach could be to exclude some files from the analysis, with the compromise that the attack could effectively be contained within them.

7 Discussion

This final section aims to interpret the results obtained, contextualize the contributions of this thesis in the landscape of software security, analyze its intrinsic limitations, and outline possible directions for future work. The analysis conducted has shown that, although attacks like BST and HUT aim to overcome human perception, they leave clear traces that can be detected through a targeted approach.

7.1 Interpretation of the Results

The results presented in Section 6 confirm the central hypothesis of this thesis: an evolutionary and differential analysis based on specific metrics is capable of successfully detecting attacks designed to be invisible to the human eye.

For BST type attacks, the W/P Ratio metric proved to be exceptionally effective. A software project, large or small, tends to maintain a relative balance between code and whitespaces, but the injection of a BST attack drastically breaks this stability, causing an anomalous surge in the ratio.

The MLL metric acts as a confirmation indicator since modern coding standards recommend code lines of a maximum of 120 characters, and the jump to a line of over 1400 characters (as in the simulated attack) represents a statistically impossible anomaly to ignore in a “healthy” code file.

For HUT type attacks, the effectiveness of the OC and IC metrics lies in their “binary nature”: their appearance in a code file, particularly within function or variable names, is almost always an anomaly. It is true that some files may legitimately contain non-ASCII characters (as discussed in the false positive analysis), but the principle of historical analysis remains valid: the sudden increase in their number within the code constitutes an alarm bell not to be underestimated.

7.2 Fragility of the Development Ecosystem

A crucial point that emerged from this thesis is the fragility of the development ecosystem in the face of these types of attacks. To verify the current state of defenses against these types of attacks, an analysis was conducted on the main code hosting platforms in order to evaluate how they manage the visualization of source code and if they implement countermeasures against visual obfuscation.

The results, shown in Table 2, reveal a fragmented and inconsistent situation. In fact, even a mature platform like *GitHub* offers partial protection: while the *soft-wrap* in Pull Requests can mitigate a BST attack, this defense is not applied when navigating the code directly in a repository. In this mode, the horizontal scroll bar can even disappear after a few seconds of inactivity, making a hidden payload almost impossible to detect.

For HUT attacks, the situation is even more critical: the software only flags the possible presence of *BiDi* control characters, but there is no warning against the use of homoglyphs or other invisible characters in the code, which represent the heart of this threat.

Other platforms, such as *npm*, *NuGet*, and *Yarn*, do not apply any form of wrapping, leaving the human reviewer completely exposed. Many other package registries, such as *PyPI* and *CocoaPods*, entirely delegate code visualization to *GitHub*, effectively inheriting the same vulnerabilities. This widespread lack of a standardized and complete defense at the infrastructure level reinforces the need for specialized analysis tools.

Table 2: Analysis of countermeasures against invisible attacks on development platforms.

Platform	BST Mitigation	HUT Mitigation
<i>GitHub</i>	Partial ¹	Partial ²
<i>RubyGems</i>	Yes	No
<i>npm</i>	No	No
<i>NuGet</i>	No	No
<i>Yarn</i>	No	No
<i>PyPI</i>	N/A ³	N/A ³
<i>CocoaPods</i>	N/A ³	N/A ³
<i>Anaconda</i>	N/A ³	N/A ³

¹ Text wrapping is active in Pull Requests, but not in the standard file view in the repository.

² A warning is shown only for BiDi control characters, but not for homoglyphs or other invisible characters.

³ The platform does not offer a native code view and refers to external repositories (typically *GitHub*).

7.3 Contributions of the Thesis

The study conducted offers several contributions to the field of SSC security. In particular, it has been shown that generic software metrics are inadequate for detecting attacks based on visual obfuscation. Instead, an approach based on the evolutionary analysis of fine-grained metrics specifically designed for these types of attacks has been proposed and validated.

An open-source tool that implements the proposed methodology, *BST_HUT_analyzer*, has also been developed and released: The tool is scalable, configurable, and can potentially be used by project maintainers to search for past tampering or to monitor new contributions.

Finally, a dataset of 91 repositories, both original and manipulated with fake backdoors, was created, which can serve as a basis for future studies.

7.4 Limitations of the Study

Despite the positive results obtained, it is essential to recognize the limitations of this study, which do not invalidate the validity of the approach, but define its boundaries and pave the way for future research.

****Nature of the Dataset**:** The validation of the tool and metrics is based on a dataset in which vulnerabilities were artificially injected: while this approach has made it possible to create a controlled and repeatable test environment, it represents an abstraction of a real-world scenario. In fact, an attacker could employ various obfuscation techniques distributed over multiple commits to minimize statistical anomalies, presenting a more complex challenge compared to the single simulated attack.

****Context for HUT Attacks**:** A limitation of the current implementation for detecting HUT attacks is the lack of sensitivity to the syntactic context: the tool flags the presence

of homoglyph or invisible characters regardless of their position. As discussed in the false positive analysis, the tool does not distinguish between a suspicious character in a function identifier and the same character in a comment or in an internationalization string. This lack, which could be overcome by integrating an AST analysis, makes human intervention necessary to validate the alarms.

* **Dependence on History**: The effectiveness of the proposed methodology is intrinsically linked to its historical analysis approach, which detects anomalies by comparing different releases of a project. This dependence on previous history constitutes a significant limitation: the method needs a sufficient number of “healthy” versions to establish a baseline. Consequently, in the early stages of a repository’s life, the tool is not very effective at detecting an attack.

* **Focus on the Programming Language**: Although the metrics were designed to be applicable to any programming language, the dataset and empirical validation focused primarily on JavaScript repositories. The metric values can vary considerably depending on the language used due to the formatting and conventions adopted. Consequently, the effectiveness and any alarm thresholds of the tool may require “recalibration” to ensure the same reliability in other programming ecosystems.

7.5 Future Work and Practical Implications

The analysis conducted and the tool developed in this thesis represent a starting point for future research and practical applications in the field of SSC security. Future directions can be divided into direct improvements to the tool and broader implications for the development ecosystem.

The main practical application of *BST_HUT_analyzer* is its integration into CI/CD systems. The tool could be configured as an automatic step that analyzes each new pull request or commit and, instead of analyzing the entire history, could compare the metrics of the modified files with their previous version in the main branch. An anomalous surge in one of the key metrics could automatically block the merge process and notify the maintainers, transforming the tool into an active defense system.

Furthermore, to overcome the identified limitations and increase the tool’s effectiveness, a list of future research directions has been drawn up:

* **Contextual Analysis via AST for HUT Attacks**: A crucial evolution is the contextual analysis of the code via the AST. This method allows understanding where a homoglyph character is used: whether in a variable or function name (critical threat) or in a simple comment (low criticality). In this way, false positives are drastically reduced, and more precise alarms are provided to reviewers.

* **Machine Learning-Based Approaches**: To overcome detection based only on sudden variations, machine learning could be applied; by training a model on a project’s history, it would learn its typical fluctuations, thus becoming capable of identifying more sophisticated and well-developed attacks.

* **Dataset Expansion with Complex Attack Scenarios**: The test dataset could be expanded to include more sophisticated attack scenarios, such as attacks distributed across multiple files, attacks that combine BST and HUT techniques, or backdoors that are only activated in combination with changes in other parts of the code.

7.6 Conclusions

This thesis addressed the problem of source code attacks, such as Blank Space Trojan (BST) and Hidden Unicode Trojan (HUT), which exploit the limits of human perception to bypass code review processes.

It has been shown that traditional software metrics are ineffective against these threats, and the proposed solution is based on an evolutionary analysis of a project's releases, using fine-grained metrics specifically designed to detect the anomalies introduced by such attacks.

While showing high effectiveness, the approach has room for improvement as discussed in section 7.5. The next step is the integration of the tool into a CI/CD pipeline, transforming it into a true defense tool that can alert maintainers in real time.

Finally, this study reaffirms the need to strengthen trust in the open-source ecosystem, and to do so, new tools are needed to assist humans and increase the security of software products and their dependencies.

References

- [1] Qwiet AI. Github hijack: What you should know. <https://qwiet.ai/github-hijack-what-you-should-know/>.
- [2] Aikido security. <https://www.aikido.dev/>.
- [3] Angular. Angular github repository. <https://github.com/angular/angular>.
- [4] Cve-2021-42574. <https://nvd.nist.gov/vuln/detail/cve-2021-42574>.
- [5] Nicholas Boucher and Ross Anderson. Trojan Source: Invisible Vulnerabilities. In *32nd USENIX Security Symposium (USENIX Security 23)*, Anaheim, CA, August 2023. USENIX Association.
- [6] Cve-2021-42694. <https://nvd.nist.gov/vuln/detail/cve-2021-42694>.
- [7] Ianstormtaylor. Superstruct github repository. <https://github.com/ianstormtaylor/superstruct>.
- [8] Delano Oliveira, Reydne Santos, Fernanda Madeiral, Hidehiko Masuhara, and Fernando Castor. A systematic literature review on the impact of formatting elements on code legibility. *Journal of Systems and Software*, 203:111728, 2023.
- [9] Tom Preston-Werner. Semantic versioning 2.0.0. <https://semver.org/>.
- [10] Redom. Redom github repository. <https://github.com/redom/redom>.
- [11] Vuejs. Vue github repository. <https://github.com/vuejs/vue>.
- [12] Wikipedia. Bidi chars. https://en.wikipedia.org/wiki/Bidirectional_text.

Acknowledgments

I would like to warmly thank my supervisors Giacomo Benedetti and Luca Caviglione for their constant guidance, patience, and teachings. A special thanks also to Prof. Giovanni Lagorio for his great availability and valuable help. A final huge thank you to my family and closest friends, who are always by my side.